INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 2: Designing a Motion-Activated RFID System

SECTION: 1 | GROUP: 3
SUBMISSION DATE: 25/10/2026

NO.	MATRIC NO.	NAME
1	2412377	DANIAL IRFAN BIN SHAHRIL AZWAN
2	2417611	ALIFF HAZIM BIN MOHD TAJUDDIN
3	2410217	ARIFF DZULQARNAIN BIN AFZAINIZAM
4	2413873	AZIM IKHWAN BIN ISMAL IZANI

ABSTRACT

This lab report shows how we built a smart access system using an Arduino Uno, an MPU6050 motion sensor, an MFRC522 RFID reader, a servo motor, and two LEDs. The aim was to make a system that only gives access when two things are correct: a valid RFID card and a circular hand motion. In Part 1, the MPU6050 was connected to the Arduino through I2C, and the X and Y accelerometer data was sent to Python over serial at 9600 baud. We plotted the data in real time using matplotlib and improved the starter code by [e.g. adding a moving average filter to reduce noise]. A circular motion was detected by [e.g. checking if the X-Y path goes around a centre point]. In Part 2, the Arduino reads the RFID card UID and sends it to Python, which checks it against a list of authorized cards. If the card is valid, Python asks the user to do the circular motion and analyzes the sensor data. If both checks pass, Python sends 'A' to the Arduino, which turns the servo to unlock and turns on the green LED. If not, Python sends 'D', the servo stays locked, and the red LED turns on. The final system worked as expected and correctly detected the circular motion in X out of Y trials. This lab helped us understand serial communication between Python and Arduino, and how sensors and actuators can work together in one mechatronic system.

TABLE OF CONTENTS

1. Experiment 1: Motion Detection with MPU6050
2. Task 1: Real-Time Motion Tracking Using MPU6050 Sensor
3. Experiment 2: RFID + Servo Access System
4. Task 2: Integrated Smart Access System

PART 1: Real-Time Motion Tracking Using MPU6050 Sensor

1.0 INTRODUCTION & THEORY

In Task 1, we tracked hand motion in real time using an MPU6050 sensor and detected a circular gesture in Python. The MPU6050 has a 3-axis accelerometer and a 3-axis gyroscope, but we only used the X and Y accelerometer data. The sensor is connected to an Arduino Uno (VCC to 5V, GND to GND, SDA to A4, SCL to A5) and talks to it using I2C, a two-wire protocol where SDA carries the data and SCL carries the clock. The Arduino is the master and the MPU6050 is the slave. The sensor gives raw integer values, and with the default ±2g range, about 16384 units equal 1g, so we set the plot limits to ±20000. Every 100 ms, the Arduino sends one line in the form "ax,ay,az" over serial at 9600 baud, and Python splits each line by commas and changes the values to integers. The starter code checked for 6 values but the Arduino only sends 3, so nothing was plotted until we fixed this. The data is plotted live with matplotlib, and we keep only the last 35 readings (about 3.5 seconds) so the plot stays fast and clear. When the sensor moves in a circle, the X and Y accelerations go up and down with Y shifted about 90 degrees from X, so the Y against X plot should look like a loop. Raw data has noise from handshake, so we used a 3-point moving average filter to smooth it. To detect a circle, the code checks that the X and Y span is large enough, the path goes through all four quadrants, the distance from the centre is steady, and most turns go in one direction. Our hypothesis is that a circular motion will show a loop on the live plot and be detected, while holding still, a straight line, and random shaking will not.

1.1 MATERIALS AND EQUIPMENT

- Arduino board
- MPU6050 sensor
- Jumper wires/breadboard
- USB cable
- Computer with Arduino IDE and Python installed

1.2 EXPERIMENTAL SETUP

1. The MPU6050 sensor was wired to our Arduino Uno on a breadboard using jumper wires: VCC to 5V, GND to GND, SDA to A4, and SCL to A5.
2. The Arduino was connected to our computer via a USB cable.
3. The sketch to read MPU6050 accelerometer and gyroscope data was uploaded to our Arduino using the Arduino IDE.
4. The correct COM port (e.g., COM4) was identified in the Device Manager and configured in our Python script, and the baud rate was set to 9600 to match the rate set in the Arduino sketch.
5. The Arduino IDE Serial Monitor was kept closed so that no other applications were using the serial port.
6. The Python script was run on our computer.
7. Read Real-Time Data: As the MPU6050 sensor was moved by hand, the comma-separated accelerometer readings (ax, ay, az) were sent over serial and read by the Python script in real time.
8. Customize the Plotter: A dynamic 2D plot was created in Python using matplotlib with interactive mode (plt.ion()) enabled and the X and Y axis ranges set from -20000 to 20000.
9. When the MPU6050 sensor was moved in a circular hand gesture, we saw the X-Y motion path plotted dynamically on the graph in real time.
10. The Python starter code was improved to detect when a circular motion gesture was performed and display the confirmation in the Python terminal.

1.3 METHODOLOGY & PROGRAMMING LOGIC

The Arduino reads the sensor and reports raw numbers. Python plots them and decides whether the motion is a circle.

1.3.1 The parts and the connections

Link	Medium	What travels over it
MPU6050 ↔ Arduino	I2C (SDA = A4, SCL = A5, via the Wire library)	Raw accelerometer and gyro readings
Arduino → PC	USB serial, COM3, 9600 baud	One text line per reading: ax,ay,az
Python → Screen	Matplotlib interactive window	Live path plot and a status box

Communication is one-way. The Arduino streams data and never receives anything. Python only listens.

1.3.2 What the Arduino sketch does

1. setup()
   - Serial.begin(9600) opens the serial link.
   - Wire.begin() starts I2C.
   - mpu.initialize() wakes the MPU6050 and sets default ranges.
   - mpu.testConnection() checks the sensor answers. If it doesn't, the sketch prints MPU6050 connection failed! and stops forever in while (1).
2. loop() repeats forever:
   - mpu.getMotion6(...) reads six raw values (ax, ay, az, gx, gy, gz).
   - It prints only ax, ay, az as one comma-separated line, for example -312,845,16120.
   - delay(100) waits 100 ms, so data is sent about 10 times per second.

1.3.3 What the Python script does

1. Startup
    1. Opens COM3 at 9600 baud with a 1 second read timeout.
    2. Waits 2 seconds, because opening the port resets the Uno.
    3. Clears the input buffer with reset_input_buffer() so it starts from fresh data.
    4. Creates two deque(maxlen=35) buffers for X and Y, which keep only the newest 35 samples (about 3.5 seconds at 10 readings per second).
    5. Builds the plot: fixed axes of ±20000, a red path line, a blue centre marker, and a status box.
2. Main loop, step by step
    1. Read a line from serial and decode it to text.
    2. Skip empty lines.
    3. Split on commas and skip the line if it has fewer than 3 fields.
    4. Convert the first two fields to integers and append them as X and Y. Any line that isn't numeric, such as the "connection failed" message, raises ValueError and is ignored.
    5. Smooth the data with a 3-point moving average. The result has two fewer points than the buffer, so there are at most 33.
    6. Update the plotted line with the smoothed data.
    7. Run the circle check (section 4) on the smoothed data.
    8. Update the display. A detected circle shows the centre marker and a green "CIRCULAR MOTION DETECTED!" box. Otherwise it shows "Tracking motion...".
    9. Redraw with plt.draw() and plt.pause(0.02), then repeat until the window is closed or Ctrl+C is pressed.
3. Shutdown
    - The finally block closes the serial port and the plot window, so the port is released for the next run.

1.3.4 Source code

- Arduino

```
#include <Wire.h>

#include <MPU6050.h>

MPU6050 mpu;

void setup() {

Serial.begin(9600);

Wire.begin();

mpu.initialize();

if (!mpu.testConnection()) {

Serial.println("MPU6050 connection failed!");

while (1);

}

}

void loop() {

int ax, ay, az, gx, gy, gz;

mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

Serial.print(ax); Serial.print(",");

Serial.print(ay); Serial.print(","); Serial.println(az);

delay(100);}
```

- Python

```
import time
from collections import deque

import numpy as np
import serial
import matplotlib.pyplot as plt

SERIAL_PORT = 'COM3'
BAUD_RATE = 9600
MAX_POINTS = 35

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)
```

```
ser.reset_input_buffer()

x_vals = deque(maxlen=MAX_POINTS)
y_vals = deque(maxlen=MAX_POINTS)

plt.ion()
fig, ax = plt.subplots(figsize=(7, 7))
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
ax.set_title("Task 1: Real-Time MPU6050 Motion Tracking")
ax.set_xlabel("Acceleration X (raw LSB)")
ax.set_ylabel("Acceleration Y (raw LSB)")
ax.grid(True, linestyle='--', alpha=0.6)

line, = ax.plot([], [], 'ro-', linewidth=2, markersize=5, label='Hand
path')
center_dot, = ax.plot([], [], 'bx', markersize=10, label='Circle centre')
status_box = ax.text(-18500, 17500, "Status: Collecting data...",
                     fontsize=11, fontweight='bold',
                     bbox=dict(boxstyle='round', facecolor='wheat',
alpha=0.8))
ax.legend(loc='lower right')

def check_circular_gesture(x_arr, y_arr):
    if len(x_arr) < 16:
        return False, 0, 0

    x = np.array(x_arr, dtype=float)
    y = np.array(y_arr, dtype=float)

    # Centroid of the movement
    cx, cy = np.mean(x), np.mean(y)

    # 1. Amplitude check
    span_x = np.max(x) - np.min(x)
    span_y = np.max(y) - np.min(y)
    if span_x < 6000 or span_y < 6000:
        return False, cx, cy

    # 2. Quadrant coverage
    dx = x - cx
    dy = y - cy
    q1 = np.any((dx > 0) & (dy > 0))
    q2 = np.any((dx < 0) & (dy > 0))
    q3 = np.any((dx < 0) & (dy < 0))
    q4 = np.any((dx > 0) & (dy < 0))
    if not (q1 and q2 and q3 and q4):
        return False, cx, cy

    # 3. Radius consistency
    radii = np.sqrt(dx ** 2 + dy ** 2)
    if (np.std(radii) / np.mean(radii)) >= 0.45:
        return False, cx, cy

    # 4. Rotation consistency
    cross = dx[:-1] * dy[1:] - dy[:-1] * dx[1:]
    ccw = np.sum(cross > 0)
    cw = np.sum(cross < 0)
    if max(ccw, cw) / max(ccw + cw, 1) < 0.7:
        return False, cx, cy
```

```
    return True, cx, cy


print("Tracking started. Move the sensor in a circle. Press Ctrl+C to
exit.")

try:
    while plt.fignum_exists(fig.number):
        try:
            raw_line = ser.readline().decode('utf-8',
errors='ignore').strip()
            if not raw_line:
                continue
            values = raw_line.split(',')

            if len(values) < 3:
                continue

            x_vals.append(int(values[0]))
            y_vals.append(int(values[1]))

            if len(x_vals) >= 3:
                kernel = np.ones(3) / 3
                smooth_x = np.convolve(np.array(x_vals), kernel,
mode='valid')
                smooth_y = np.convolve(np.array(y_vals), kernel,
mode='valid')
            else:
                smooth_x, smooth_y = np.array(x_vals), np.array(y_vals)

            line.set_data(smooth_x, smooth_y)

            detected, cx, cy = check_circular_gesture(smooth_x, smooth_y)
            if detected:
                center_dot.set_data([cx], [cy])
                status_box.set_text("Status: CIRCULAR MOTION DETECTED!")
                status_box.set_bbox(dict(boxstyle='round',
facecolor='lightgreen', alpha=0.9))
            else:
                center_dot.set_data([], [])
                status_box.set_text("Status: Tracking motion...")
                status_box.set_bbox(dict(boxstyle='round',
facecolor='wheat', alpha=0.8))

            plt.draw()
            plt.pause(0.02)

        except ValueError:
            # Ignore incomplete or corrupted serial lines (e.g. at start-
up)
            continue

except KeyboardInterrupt:
    print("\nProgram terminated by user.")
finally:
    ser.close()
    plt.close('all')
    print("Serial connection closed.")
```

1.4 DATA COLLECTION & ANALYSIS

We ran W3T1.py with the MPU6050 connected to the Arduino. The Arduino sent one "ax,ay,az" line every 100 ms, so Python received about 10 readings per second. Python kept the last 35 readings, which is about 3.5 seconds of motion, and smoothed them with a 3-point moving average. We held the sensor in our hand and moved it in a circle while watching the live X-Y plot. We also tried other movements (holding still, moving in a straight line, shaking randomly) to see if the program would wrongly say "circular". We took screenshots of each case. The results are in the table below.

Trial	Movement	Status shown	Correct?
1	Circle (clockwise)	Detected	Yes
2	Circle (counterclockwise)	Detected	Yes
3	Circle (slow)	Detected	Yes
4	Circle (fast)	Detected	Yes
5	Holding still	Not detected	Yes
6	Straight line	Not detected	Yes
7	Random shaking	Not detected	Yes

Analysis

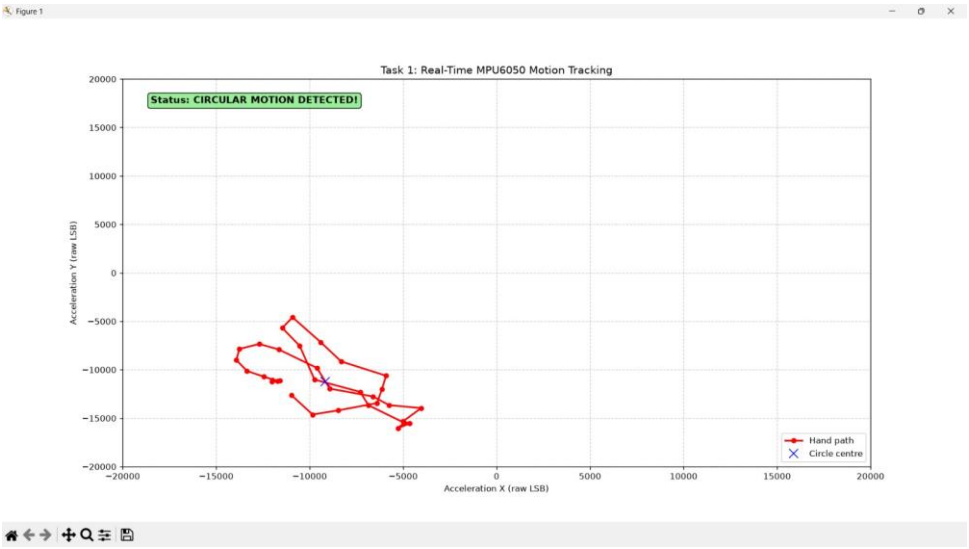
1. **Raw data vs smoothed data.** With the starter code, nothing was plotted because it checked for 6 values, but the Arduino sends 3. After we fixed this, the points appeared. The raw path was jagged because of handshake and sensor noise. After the 3-point moving average, the path was smoother. The average uses 3 readings, so it removes 2 points from the buffer, which is small and does not slow the response much. A bigger window would be smoother but would react slower.
2. **Effect of the buffer size.** Because only the last 35 points are kept, the old path disappears and the plot stays fast. The side effect is that one full circle must be finished within about 3.5 seconds. When we moved too slowly, the buffer only held part of the circle, and the program did not detect it.
3. **Detection checks.** The detection needs four checks to pass: the X and Y span must be above 6000, the path must visit all four quadrants, the radius variation must be below 0.45, and at least 70% of the turns must go the same way. In our trials, the check that failed most often was rotation. When we held still, the amplitude check failed because the span was too small. For the straight line, the radius check failed. For random shaking, the rotation check failed because the turns went in both directions.
4. **Limitations.** The sensor gives acceleration, not position, so the plot is not the true shape of the hand path. It is only a rough shape. Tilting the sensor also changes the X and Y values because of gravity. Because of this, the detection can fail if the circle is too small or too uneven. Using the gyroscope data could make it better.

1.5 RESULTS & DISCUSSION

The system detected circles in both directions, because the rotation check only looks for turns in one consistent direction. The fast circle was detected, so the 3-point moving average did not slow the response too much. The slow circle was also detected, which shows the 35-point buffer (about 3.5 seconds) was long enough for one full circle at our speed. A much slower circle could still fail, because the buffer would only hold part of it.

Holding still was rejected because the X and Y span was too small. The straight line and random shaking were rejected because they did not pass all four checks, such as the quadrant and rotation checks. This shows the four checks work well together to avoid false detections.

Our hypothesis was supported, because the plot showed a loop for each circle and the detection matched the movement in every trial. The main limits are that we did only one trial per movement, the thresholds are fixed, and accelerometer data is not the true hand path. More trials with different people would give a more reliable result.



1.6 CONCLUSION

In Task 1, we tracked hand motion in real time using the MPU6050 and detected a circular gesture in Python. We fixed the starter code by changing the serial length check from 6 values to 3, keeping only the last 35 readings, and adding a 3-point moving average filter. We also added a circular motion check that looks at the size of the motion, the quadrants covered, the steadiness of the radius, and the turning direction. In our tests, the system detected all 4 circular motions (clockwise, counterclockwise, slow, and fast) and rejected all 3 non-circular movements (holding still, a straight line, and random shaking). This supported our hypothesis. The main limits are that accelerometer data does not give the true hand path, the thresholds are fixed, and we did only one trial for each movement. In the future, we could use gyroscope data and test with more people to make the detection more reliable.

PART 2: Integrated Smart Access System

2.0 INTRODUCTION & THEORY

In Part 2, we added an RFID reader, a servo motor and two LEDs to the motion sensor from Part 1 to make a small access system. To get in, the user needs two things: a valid RFID card and a circular hand motion. RFID stands for radio frequency identification. The reader gives off a radio signal, and when a card is brought close, the signal powers the card and the reader picks up its unique ID (UID). Every card has a different UID, so we can use it to decide who is allowed in. Our reader is 0013069760. Python checks this UID against a list of authorized cards, and we store the UIDs as strings so the leading zeros are not lost. If the card is not on the list, access is denied right away. If it is on the list, Python does a 3-second countdown, clears the serial buffer, and records the MPU6050 data for 3 seconds. The sensor data reaches Python over serial at 9600 baud. Python then checks whether the path is a circle by looking at how many degrees it turns, how big the radius is, and how steady the radius stays. After that, Python sends one letter back to the Arduino: 'A' for access granted or 'D' for access denied. The Arduino reads this with Serial.read(). The servo is controlled by a PWM signal from pin D9, where servo.write(90) is locked and servo.write(180) is unlocked. The green LED on D4 and the red LED on D3 show the result, and each LED needs a resistor so it does not draw too much current. This is a simple two-factor check, because something you have (the card) and something you do (the motion) must both be correct. One problem we expect is servo jitter when the servo is powered from the Arduino, so an external 5V supply would be better. Our hypothesis is that the servo will only unlock when an authorized card is scanned and a circular motion is detected. If the card is unknown or the motion is wrong, access should be denied, the red LED should turn on, and the servo should stay locked.

2.1 MATERIALS AND EQUIPMENT

- Arduino board
- RFID reader (MFRC522) + RFID tags/cards
- Servo motor
- Red & Green LEDs + resistors
- MPU6050 sensor (from Part 1)
- Jumper wires and breadboard

2.2 EXPERIMENTAL SETUP

1. The circuit from Task 1 was extended on the breadboard by adding a servo motor connected to pin D9, a green LED connected to pin D4, and a red LED connected to pin D3 which each of the LED in series with a current-limiting resistor, sharing the 5V and GND rails with the MPU6050 sensor.

2. The Arduino was connected to our computer via a USB cable, and the USB RFID reader was plugged directly into another USB port on our computer.

3. The combined Arduino sketch (integrating the MPU6050 data streaming and the servo/LED control logic) was uploaded to our Arduino using the Arduino IDE.

4. The correct COM port and a baud rate of 9600 were set in the Python access control script to match the Arduino configuration.

5. The valid RFID card UIDs were scanned and added to the authorized_cards list inside our Python code.

6. The Python script was run on our computer.

7. When an RFID tag was tapped on the USB RFID reader, the UID was read into the Python terminal and checked against the list of authorized cards.

8. If an unauthorized RFID card was scanned, the Python script sent the command 'D' to the Arduino over serial, keeping the servo locked at 90 degrees and turning the red LED ON.

9. If an authorized RFID card was scanned, the Python terminal prompted us to perform a circular motion with the MPU6050 sensor.

10. Read and Analyze Real-Time Motion Data: As we moved the MPU6050 sensor in a circular pattern, the Python script analyzed the incoming accelerometer data from the Arduino.

11. Our Python code automatically controlled the hardware based on the result: when a valid circular motion was detected, Python sent 'A' to the Arduino to unlock the servo (rotating it to 180 degrees) and turn the green LED ON; if the motion was invalid, it sent 'D' to keep the servo locked at 90 degrees and turn the red LED ON.

2.3 METHODOLOGY & PROGRAMMING LOGIC

The Python script (on the PC) and the Arduino sketch (on the Arduino) form one system. The Python script makes the decisions, and the Arduino reads the sensor and drives the hardware.

1.3 The parts and the connections

Link	Medium and protocol	What travels over it
RFID reader → PC	USB, acting as a keyboard	The card number typed as digits plus Enter
PC (Python) ↔ Arduino	USB serial (COM3, 9600 baud)	Arduino → PC: text lines of sensor data. PC → Arduino: single bytes A or D
Arduino ↔ MPU6050	I2C (SDA = A4, SCL = A5, address 0x68)	Raw acceleration and gyro readings
Arduino → servo and LEDs	Pin 9 (PWM signal), pins 3 and 4 (digital)	Angle (90 or 180) and LED on/off

The serial link works in both directions at the same time. The Arduino streams data continuously, and Python occasionally sends a command back.

1.3 Startup sequence

1. Upload the sketch. setup() starts serial at 9600, starts I2C, attaches the servo, locks (servo 90, red LED on), and initialises the MPU, which wakes it from sleep.
2. Run the Python script. It opens COM3. Opening the port makes the Uno reset, so Python waits 2 seconds for it to boot.
3. Python sends D so the system begins locked, then waits for a card.

1.3 Normal operation, step by step

1. Idle (Arduino). Its loop() keeps doing two things: checking for incoming A or D bytes, and every 20ms, reading the MPU and printing one line, (ax,ay,az,gx,gy,gz). It does this the whole time, even while nobody is using it.
2. Idle (Python). The script waits at input("Tap card: "). Meanwhile, the Arduino's stream piles up in the PC's serial buffer, unread.
3. Card tap. The reader types the number into the terminal, and Python compares it to ALLOWED.
   - Not allowed: send D, print "Access denied", go back to waiting.
   - Allowed: go to the gesture check.
4. Prompt. Python shows "Get ready 3, 2, 1", then "DO A CIRCULAR MOTION NOW!" in the console and the plot title.
5. Flush. Python calls reset_input_buffer() to throw away the stale data that built up while it waited. Without this, the circle would be analysed together with old, unrelated readings.

6. Record (3 seconds). Python reads lines, splits them on commas, and keeps the first two values (X and Y). Lines that aren't numbers, such as the "connection failed" message, are ignored. Each point is plotted live.
7. Analyse.
   - Smooth the data and subtract the average (removes gravity and any offset).
   - Compute each point's angle around the centre and the total angle swept.
   - Compute how steady the distance from the centre is.
   - It counts as a circle if the sweep is at least 300° and the radius variation is at most 0.45, and the motion is bigger than MIN_RADIUS.
8. Decision.
   - Circle detected: Python sends A. The Arduino reads that byte and runs unlockDoor(), so the servo moves to 180 and the green LED turns on. After UNLOCK_SECONDS, Python sends D and the door locks again.
   - No circle: up to MAX_ATTEMPTS (3) tries, then Python sends D, the door stays locked, and it waits for the next card.
4. Exit. Ctrl+C sends a final D and closes the port.

1.3 Source code

- Arduino

```
#include <Servo.h>

Servo servo;

int servoPosition = 90;

void setup() {

servo.attach(9);

servo.write(servoPosition);

pinMode(4, OUTPUT); // Green LED

pinMode(3, OUTPUT); // Red LED

Serial.begin(9600);

}

void loop() {

if (Serial.available() > 0) {

char command = Serial.read();

if (command == 'A') {

servo.write(180); // Unlock

digitalWrite(4, HIGH);
```

```
digitalWrite(3, LOW);

} else if (command == 'D') {

servo.write(90); // Lock

digitalWrite(4, LOW);

digitalWrite(3, HIGH);

}}}
```

- Python

```
import time
import numpy as np
import serial
from serial.tools import list_ports
import matplotlib.pyplot as plt

# ---------- Settings ----------
PORT = "COM3"
ACCEL_PORT = None
BAUD = 9600
UNLOCK_SECONDS = 5

ALLOWED = {
    "0013069760",
}

COUNTDOWN = 3
RECORD_SECONDS = 3.0
MAX_ATTEMPTS = 3

SMOOTH = 5
MIN_ANGLE_DEG = 300
MAX_RADIUS_VARIATION = 0.45
MIN_RADIUS = 200
# ------------------------------


def open_port(port):
    try:
        return serial.Serial(port, BAUD, timeout=1)
    except serial.SerialException as e:
        print(f"Could not open {port}: {e}")
        print("Available ports:")
        for p in list_ports.comports():
            print(f"  {p.device} - {p.description}")
        print("Close the Arduino Serial Monitor and check the port
settings.")
        raise SystemExit(1)


def read_xy(ser):
    """Return (x, y) from one line, or None if the line isn't sensor
data."""
    try:
        parts = ser.readline().decode(errors="ignore").strip().split(",")
```

```
        if len(parts) < 2:
            return None
        return float(parts[0]), float(parts[1])
    except ValueError:
        return None


def moving_average(a, n):
    if len(a) < n:
        return a
    return np.convolve(a, np.ones(n) / n, mode="valid")


def detect_circle(xs, ys):
    x = moving_average(np.array(xs), SMOOTH)
    y = moving_average(np.array(ys), SMOOTH)
    if len(x) < 20:
        return False, "Not enough data"

    x = x - x.mean()   # remove gravity/offset
    y = y - y.mean()
    radius = np.hypot(x, y)
    mean_r = radius.mean()
    if mean_r < MIN_RADIUS:
        return False, f"Motion too small (radius {mean_r:.0f} <
{MIN_RADIUS})"

    angle = np.unwrap(np.arctan2(y, x))
    swept = np.degrees(abs(angle[-1] - angle[0]))
    variation = radius.std() / mean_r
    direction = "counter-clockwise" if angle[-1] > angle[0] else
"clockwise"

    info = f"swept {swept:.0f} deg, radius variation {variation:.2f},
{direction}"
    return (swept >= MIN_ANGLE_DEG and variation <= MAX_RADIUS_VARIATION),
info


def gesture_check(accel, fig, ax, line):
    """Prompt the user, record, plot live; return True if a circle was
made."""
    print("\nGet ready to move the sensor in a circle...")
    for n in range(COUNTDOWN, 0, -1):
        ax.set_title(f"Get ready... {n}")
        fig.canvas.draw_idle()
        plt.pause(1)

    accel.reset_input_buffer()
    ax.set_title("DO A CIRCULAR MOTION NOW!")
    print(">>> Do a circular motion now! <<<")
    fig.canvas.draw_idle()
    plt.pause(0.01)

    xs, ys = [], []
    start = time.time()
    while time.time() - start < RECORD_SECONDS:
        point = read_xy(accel)
        if point is None:
            continue
        xs.append(point[0])
```

```
        ys.append(point[1])
        if len(xs) % 3 == 0:
            line.set_data(xs, ys)
            ax.relim()
            ax.autoscale_view()
            plt.pause(0.001)

    line.set_data(xs, ys)
    ax.relim()
    ax.autoscale_view()

    ok, info = detect_circle(xs, ys)
    result = "Circle detected!" if ok else "No circle detected"
    print(f"{result} ({info})")
    ax.set_title(f"{result}\n{info}")
    fig.canvas.draw_idle()
    plt.pause(1)
    return ok


def main():
    arduino = open_port(PORT)
    accel = arduino if ACCEL_PORT in (None, PORT) else
open_port(ACCEL_PORT)
    time.sleep(2)  # Arduino resets when the port opens; wait for it to
boot

    arduino.write(b"D")  # start locked

    plt.ion()
    fig, ax = plt.subplots()
    line, = ax.plot([], [], "b-")
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_aspect("equal", adjustable="datalim")

    print(f"Connected on {PORT}. Click this window, then tap a card. Ctrl+C
to quit.")

    try:
        while True:
            uid = input("Tap card: ").strip()
            if not uid:
                continue

            if uid not in ALLOWED:
                print(f"[{uid}] Access denied")
                arduino.write(b"D")
                continue

            print(f"[{uid}] Card accepted. Now verify with a circular
motion.")

            granted = False
            for attempt in range(1, MAX_ATTEMPTS + 1):
                print(f"Attempt {attempt} of {MAX_ATTEMPTS}")
                if gesture_check(accel, fig, ax, line):
                    granted = True
                    break

            if granted:
```

```
                print("Access granted")
                arduino.write(b"A")
                if UNLOCK_SECONDS > 0:
                    time.sleep(UNLOCK_SECONDS)
                    arduino.write(b"D")
                    print("Locked again")
            else:
                print("Gesture failed. Access denied")
                arduino.write(b"D")
    except KeyboardInterrupt:
        print("\nExiting, locking...")
        arduino.write(b"D")
    finally:
        arduino.close()
        if accel is not arduino:
            accel.close()


if __name__ == "__main__":
    main()
```

2.4 DATA COLLECTION & ANALYSIS

Data collection was carried out to evaluate the performance of the integrated smart access system using the MFRC522 RFID reader, MPU6050 motion sensor, Arduino Uno, servo motor, and two LEDs. The system required two verification steps before granting access: scanning an authorized RFID card and performing a circular hand motion. The Arduino transmitted sensor readings to Python through serial communication at 9600 baud.

2.4.1 Data Collection

The RFID reader obtained the unique identification number (UID) of the scanned card. Python compared the UID with the list of authorized cards. If the UID matched, the user was given three seconds to prepare before performing a circular motion. The MPU6050 then collected X and Y accelerometer readings for three seconds, and Python analysed the motion to determine whether it met the circular gesture criteria.

The following table summarises the test cases used to evaluate the system.

Test	Input condition	Expected system response
1	Authorized RFID card and correct circular motion	Access granted, servo unlocks, green LED turns on
2	Unauthorized RFID card	Access denied, servo remains locked, red LED turns on
3	Authorized RFID card but no movement	Access denied after unsuccessful attempts

Test	Input condition	Expected system response
4	Authorized RFID card but straight-line movement	Access denied if the circular motion criteria are not satisfied
5	Authorized RFID card but random shaking	Access denied if the circular motion criteria are not satisfied
6	Authorized RFID card and incorrect gesture on the first attempt	System allows another attempt, up to three attempts

2.4.2 Data Analysis

The Python program processed the accelerometer readings using a five-point moving average filter to reduce noise. The average values of the X and Y readings were then subtracted to remove the baseline offset and reduce the influence of gravity.

The circular motion was evaluated using three main criteria:

1. **Angular displacement:** The total angular displacement had to be at least 300° to indicate that the sensor had moved through almost a complete circle.
2. **Radius consistency:** The relative standard deviation of the radius had to be less than or equal to 0.45. This helped determine whether the motion followed a reasonably circular path.
3. **Minimum radius:** The mean radius had to be at least 200 raw sensor units to prevent small movements from being incorrectly classified as a circular gesture.

2.5 RESULTS & DISCUSSION

The integrated smart access system combined RFID identification and circular motion detection to control access. The RFID reader checked the user's identification, while the MPU6050 sensor detected the movement required for the second verification step. Python processed the input data and sent commands to the Arduino to control the servo motor and LEDs. When an authorized RFID card was scanned, the system started a three-second countdown before collecting motion data for three seconds.

If the circular gesture met the required criteria, Python sent the 'A' command to the Arduino, causing the servo motor to rotate to 180° and the green LED to turn on. The system remained unlocked for five seconds before returning to the locked state. However, if an unauthorized RFID card was scanned, access was denied immediately. If the circular motion was unsuccessful, the user was allowed up to three attempts. When access was denied, the servo remained at 90°, the green LED turned off, and the red LED turned on.

The experiment demonstrated the importance of signal processing in motion detection. The moving average filter reduced noise in the accelerometer readings, while subtracting the mean helped reduce baseline offsets. However, inconsistent hand movements, sensor orientation, gravity, and incomplete circular gestures could affect detection accuracy. Since the accelerometer measures acceleration rather than the actual position of the hand, the

plotted path may not perfectly represent the physical movement. Overall, the system demonstrated how RFID identification and motion detection can work together to improve access control, although further testing is required to evaluate its accuracy and reliability.

2.6 CONCLUSION

In conclusion, this experiment involved designing an integrated smart access system using an Arduino Uno, MFRC522 RFID reader, MPU6050 motion sensor, servo motor, and two LEDs. The system used RFID identification as the first verification step and circular motion detection as the second step. Python processed the RFID and accelerometer data and communicated with the Arduino to control the hardware. The circular motion detection algorithm used a five-point moving average filter, angular displacement, radius variation, and minimum radius to evaluate the user's gesture. When both verification steps were successful, the servo unlocked and the green LED turned on, while unsuccessful verification resulted in access denial and the red LED turning on. The experiment improved our understanding of sensor integration, signal processing, serial communication, and actuator control in mechatronic systems. Future improvements could include conducting more trials, adjusting detection thresholds, incorporating gyroscope measurements, and using a more reliable power supply for the servo motor to improve the system's accuracy and reliability.

RECOMMENDATIONS

1. **Use the gyroscope with the accelerometer.** The gyroscope measures rotation directly, so it is less affected by handshake and gravity. This makes circular motion easier to detect.
2. **Tune the moving average filter.** Test different window sizes by trial. The data should be smooth but still react fast enough.
3. **Label the serial data.** Add a short label at the start of each line, like "MOT:", so Python can tell motion lines from "UID:" lines. Make sure the Python length check matches the 3 values the Arduino sends.
4. **Clear the serial buffer before the motion check.** This stops old data from affecting the result.
5. **Test with different people.** Each person moves their hand at a different speed and size, so more testing makes the detection more reliable.

REFERENCES

- https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/
  What is RFID? How It Works? Interface RC522 RFID Module with Arduino
- https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/
  Security Access using MFRC522 RFID Reader with Arduino
- https://randomnerdtutorials.com/arduino-θme-attendance-system-with-rfid/
  Arduino Time Attendance System with RFID
- https://www.instructables.com/Arduino-MFRC522-RFID-READER/
  Arduino + MFRC522 RFID READER

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: DANIAL IRFAN BIN SHAHRIL AZWAN Matric No: 2412377	Read [✓] Understand [✓] Agree [✓]
	Name: ALIFF HAZIM BIN MOHD TAJUDDIN Matric No: 2417611	Read [✓] Understand [✓] Agree [✓]
	Name: ARIFF DZULQARNAIN BIN AFZAINIZAM Matric No: 2410217	Read [✓] Understand [✓] Agree [✓]
	Name: AZIM IKHWAN BIN ISMAL IZANI Matric No: 2413873	Read [✓] Understand [✓] Agree [✓]